

BukScan

Kompletna dokumentacja projektu — pytania i odpowiedzi od A do Z

ARCHITEKTURA · SCRAPER · ARBITRAŻ · OGRANICZENIA · PLANY

Skaner arbitrażu bukmacherskiego na polskim rynku
Czerwiec 2026

Spis treści

1. Wprowadzenie i cel projektu
2. Matematyka arbitrażu bukmacherskiego
3. Mapa funkcji — opis każdej strony aplikacji
4. Architektura systemu i tryby wdrożenia
5. Scraper kursów — pełny opis techniczny
6. Silnik arbitrażu — algorytmy i podatek
7. System powiadomień i alertów
8. Warstwa danych (Supabase) i bezpieczeństwo
9. API aplikacji
10. Zmienne środowiskowe
11. Stos technologiczny — pełna lista
12. Co udało się zrobić (zakres działający)
13. Co nie udało się zrobić / czego zabrakło
14. Główne ograniczenia projektu
15. Decyzje produktowe i ich uzasadnienie
16. Plany na przyszłość
17. Podsumowanie

1. Wprowadzenie i cel projektu

P Co to jest BukScan?

BukScan to aplikacja webowa (Next.js, App Router, TypeScript) do wykrywania okazji arbitrażu bukmacherskiego (surebetów) na polskim rynku bukmacherskim. Pobiera kursy bezpośrednio ze stron 10 polskich bukmacherów metodą web scrapingu (Playwright), porównuje je między sobą i wskazuje sytuacje, w których postawienie odpowiednio rozłożonych stawek na wszystkie możliwe wyniki meczu gwarantuje zysk niezależnie od wyniku sportowego.

P Dlaczego nie użyto gotowego API bukmacherskiego (np. The Odds API)?

Projekt zaczynał się od integracji z The Odds API (widoczne w kodzie jako `src/lib/odds-api.ts` i tryb bez klucza = demo), ale to API nie obejmuje polskich bukmacherów (STS, Superbet, Fortuna itd.) — tylko zagraniczne marki. Skoro celem jest rynek polski, jedynym sposobem dotarcia do realnych kursów krajowych firm było napisanie własnego scrapera DOM. Adapter do The Odds API pozostał w kodzie jako alternatywne źródło danych (włączane ustawieniem zmiennej `ODDS_API_KEY`), ale w produkcji aplikacja korzysta ze scrapera.

P Dla kogo jest ta aplikacja?

Dla osób grających w zakłady wzajemne w Polsce, które chcą:

- automatycznie wykrywać surebety bez ręcznego porównywania kursów u 10 bukmacherów na raz,
- porównywać, który bukmacher daje aktualnie najwyższy kurs na dany wynik,
- mieć kalkulator do ręcznego liczenia podziału stawek przy własnoręcznie znalezionych okazjach,
- śledzić historię zakładów (dziennik) i realną rentowność po odliczeniu 12% podatku,
- otrzymywać powiadomienia e-mail o zbliżających się meczach lub o przekroczeniu zadanego kursu.

2. Matematyka arbitrażu bukmacherskiego

P Na czym polega arbitraż (surebet)?

Arbitraż zachodzi, gdy suma odwrotności najlepszych dostępnych kursów na wszystkie wyniki tego samego zdarzenia (u różnych bukmacherów) jest mniejsza niż 1:

$$1/\text{kurs_1} + 1/\text{kurs_X} + 1/\text{kurs_2} < 1$$

Zysk procentowy: $(1 / \text{suma_odwrotności} - 1) \times 100$. W praktyce marża wbudowana przez bukmacherów sprawia, że ta suma jest zwykle > 1 — okazja powstaje tylko wtedy, gdy różni bukmacherzy wycenią to samo zdarzenie inaczej.

P Jak dokładnie liczone są stawki na poszczególnych wynikach?

Funkcja `calculateArbitrageStakes` (`src/lib/arbitrage.ts`) dzieli całkowitą stawkę proporcjonalnie do odwrotności kursu każdego wyniku: $\text{stawka_i} = \text{stawka_calkowita} \times (1/\text{kurs_i}) / \text{suma_odwrotnosci}$. Taki rozkład gwarantuje identyczny zwrot bez względu na to, który wynik faktycznie zajdzie.

P Jak działa wykrywanie arbitrażu w skali całej bazy wydarzeń?

`findArbitrageForEvent` przegląda każdy rynek (market) danego wydarzenia, bierze najlepszy kurs każdego możliwego wyniku (niezależnie od tego, który bukmacher go daje), przelicza go na kurs netto (po podatku) i sprawdza warunek arbitrażu. `findAllArbitrages` robi to dla wszystkich wydarzeń i sortuje wynik malejąco po procentowym zysku. Każda znaleziona okazja dostaje deterministyczne ID w formacie `{eventId}-{marketKey}`.

P Czym jest „watchlista” (near-arbitrage) i jak liczony jest kurs docelowy?

`findNearArbitrageForEvent` szuka wydarzeń, których suma odwrotności kursów netto jest tylko odrobinę powyżej 1 (domyślny próg: 4%, w warstwie produktowej `data-service.ts` dodatkowo ograniczony do 3%). Dla każdego wyniku liczy, ile pozostałej "implikowanej probability" zostaje po odjęciu jego własnego udziału, a następnie wylicza $\text{targetNetOdds} = 1 / (1 - \text{remainingImpliedProbability})$ — minimalny kurs netto, który zamieniłby sytuację w pełny surebet. Funkcja `grossOddsForNet` przelicza to z powrotem na kurs brutto (uwzględniając, czy dany bukmacher jest bez podatku), żeby użytkownik widział realną liczbę, jaką musi zobaczyć na stronie bukmachera. Wybierany jest wariant wymagający najmniejszej zmiany kursu (`requiredOddsChangePercent`).

P Jak liczona jest marża bukmachera?

`calculateMargin(odds)` sumuje odwrotności wszystkich kursów oferowanych przez jednego bukmachera na dany rynek i odejmuje 100% — to jest jego rzeczywisty narzut. W

`getBookmakerMargins` (`data-service.ts`) margże są agregowane per bukmacher i per kategoria sportu (piłka klubowa, Mistrzostwa Świata, koszykówka), z odrzuceniem wartości poza zakresem [-2%, 25%] jako artefaktów błędnego parsowania.

P Co to jest "hedge stake" i gdzie jest używany?

`calculateHedgeStake(originalStake, originalOdds, hedgeOdds)` liczy, jaką stawkę trzeba postawić na przeciwny wynik u innego bukmachera, żeby zabezpieczyć (zhedgować) zakład już postawiony — niezależnie od tego, czy pierwotna sytuacja była pełnym surebetem. To pomocnicza funkcja matematyczna w bibliotece arbitrażu, dostępna do wykorzystania w kalkulatorze/strategiach.

3. Mapa funkcji — opis każdej strony aplikacji

Strona (route)	Opis
<code>/</code> — Dashboard	Szybki przegląd: liczba surebetów, rynki bliskie progu, średni profit, lista nadchodzących wydarzeń, spersonalizowane powitanie zalogowanego użytkownika.
<code>/arbitrage</code>	Pełna lista aktualnych surebetów z rozkładem stawek i gwarantowanym zwrotem po podatku, plus sekcja watchlisty (near-arbitrage).
<code>/events</code> i <code>/events/[id]</code>	Przeglądarka wszystkich zebranych wydarzeń z filtrowaniem po sporcie/lidze i sortowaniem po czasie startu; widok szczegółowy pokazuje pełne zestawienie kursów wszystkich bukmacherów obok siebie dla danego meczu.
<code>/kalkulator</code>	Ręczny kalkulator arbitrażu — użytkownik wpisuje własne kursy i bukmacherów, kalkulator liczy rozkład stawek i ewentualny gwarantowany zysk; wynik można zapisać do dziennika.
<code>/margins</code>	Tabela realnie obliczonych marż każdego bukmachera na podstawie faktycznie zescrapowanych kursów (nie deklaracji marketingowych), z podziałem na piłkę nożną klubową, Mistrzostwa Świata i koszykówkę.
<code>/account</code>	Panel użytkownika z zakładkami: Przegląd (dziennik + statystyki), Bukmacherzy (lista posiadanych kont), Powiadomienia (ustawienia e-mail), Salda (wirtualne saldo na koncie u każdego bukmachera), Bezpieczeństwo, Subskrypcja (plan premium).
<code>/notifications</code>	Historia powiadomień systemowych: nowe surebety, okazje wchodzące w próg (pre-surebet), zakończone zakłady, wygasłe okazje.
<code>/groups</code>	Tabele grup Mistrzostw Świata 2026 (dane realne, wpisane ręcznie — patrz sekcja 5) wraz z meczami i kursami zebranymi dla reprezentacji.
<code>/strategies</code>	Edukacyjne narzędzia: kalkulator Value Bet (porównanie kursu bukmachera z własnym oszacowaniem prawdopodobieństwa, Expected Value, rekomendacja stawki według kryterium Kelly'ego) i inne strategie zakładowe.
<code>/roadmap</code>	Wizualna roadmapa projektu z podziałem na 6 faz, stanem ukończenia każdego zadania i celami biznesowymi (liczba bukmacherów, opóźnienie danych, liczba sportów).
<code>/login</code> , <code>/register</code> , <code>/auth/callback</code>	Logowanie e-mail/hasło i Google OAuth (Supabase Auth), strona callback obsługuje powrót z OAuth.
<code>/settings</code>	Przekierowanie/skrót do ustawień w panelu konta.

P Czym jest kalkulator Value Bet w `/strategies` ?

To narzędzie nie liczy arbitrażu, a tzw. "value betting": użytkownik podaje kurs bukmachera i własne oszacowanie prawdopodobieństwa wygranej, a kalkulator liczy "kurs sprawiedliwy"

($100/\text{prawdopodobieństwo}$), Expected Value ($EV = \text{probability} \times \text{kurs} - 1$) oraz rekomendowaną stawkę według kryterium Kelly'ego: $\text{kelly} = \max(0, (\text{kurs} \times p - 1) / (\text{kurs} - 1)) \times \text{stawka}$. To jedyny fragment Fazy 4 roadmapy ("Kelly Criterion kalkulator"), który już został zaimplementowany — wcześniej niż reszta tej fazy.

P Co to za dane w sekcji Mistrzostw Świata 2026 (`/groups`)?

Dane tabel grupowych i wyników pierwszej kolejki są wpisane ręcznie w `src/lib/world-cup-groups.ts` , a nie scrapowane — w komentarzu w kodzie autor wyjaśnia, że nie ma wiarygodnego źródła do automatycznego scrapowania wyników turnieju, a zgadywanie przypisania grup byłoby tym samym problemem co "fałszywy arbitraż": pewne siebie, ale błędne dane. Plik zawiera też słownik aliasów nazw reprezentacji (np. „Korea Południowa” ↔ „Korea Republic” / „South Korea”), żeby dopasować je do nazw używanych przez scrapowanych bukmacherów.

4. Architektura systemu i tryby wdrożenia

P Czy aplikacja działa jako jeden serwer, czy to coś bardziej złożonego?

To **hybrydowe wdrożenie złożone z dwóch równoległych trybów**, co nie jest opisane w README, ale wynika z plików konfiguracyjnych w repo:

- **Pełny serwer Next.js (SSR)** — `ecosystem.config.cjs` uruchamia `next start -p 3002` pod PM2 na VPS, obok innych usług (np. n8n). W tym trybie każde żądanie strony liczy arbitraż na żywo, scraper działa automatycznie w tle (instrumentation hook), a API `/api/sync` jest dostępne.
- **Statyczny eksport (frozen snapshot)** — `scripts/build-static.mjs` + `STATIC_EXPORT=1` w `next.config.ts` generują folder `out/` (widoczny w repo, plus spakowany `out.zip`), który można wgrać przez FTP na zwykły hosting (Hostido — shared hosting bez Node.js). Ten build tymczasowo usuwa `force-dynamic` ze stron, generuje statyczne parametry dla `/events/[id]` i chowa cały katalog `src/app/api` (route handlers nie da się wyeksportować statycznie), po czym przywraca oryginalne pliki źródłowe niezależnie od tego, czy build się powiódł. Wersja statyczna jest zamrożonym zdjęciem danych z chwili builda — nie odświeża się sama.
- **Osobny proces workera alertów** — `ecosystem.alerts.config.cjs` uruchamia `scripts/alerts-worker.ts` (przez `tsx`) jako niezależny proces PM2 na VPS. To jedyny proces, który musi działać 24/7, żeby alerty e-mail funkcjonowały, niezależnie od tego, czy frontend jest hostowany statycznie czy jako pełny serwer SSR.

To pokazuje realny kompromis wdrożeniowy: docelowy/tańszy hosting frontowy (Hostido, statyczny FTP) nie obsługuje Node.js, więc logika serwerowa (scraper, alerty, API) musi żyć osobno na VPS. Frontend i logika danych są więc rozłączone na produkcji — inaczej niż sugeruje czysty model "SSR z README".

P Co robi `scripts/alerts-worker.ts`?

To standalone proces Node (uruchamiany przez `tsx`, bez Next.js), który w pętli wywołuje `checkAndSendMatchAlerts()` i `checkAndSendEventAlerts()` z `src/lib/match-alerts.ts` — te same funkcje, które w trybie pełnego serwera SSR wywołuje wewnętrzny harmonogram `startMatchAlertScheduler()` z `instrumentation-node.ts` (co 60 sekund). Worker istnieje, żeby alerty działały także wtedy, gdy frontend jest eksportowany statycznie i nie ma żadnego działającego procesu Next.js.

P Jak działa harmonogram automatycznego scrapingu (instrumentation hook)?

`src/instrumentation-node.ts`, ładowany przez Next.js `instrumentation.ts` tylko w runtime Node, wystawia `startAutoScrape()`: pierwsze uruchomienie następuje 15 sekund po starcie serwera, ale tylko jeśli plik `scraped-data.json` jest starszy niż interwał (domyślnie 480 minut, minimum wymuszone na 30 minut) — żeby deploy zaraz po ręcznym scrapingu nie wywoływał go ponownie. Globalne flagi

(`globalThis.__bukAutoScrape` , `__bukScraping`) gwarantują, że tylko jeden harmonogram działa na proces i że scraper nigdy nie odpala się dwa razy naraz (współdzielone z endpointem `/api/sync`).

5. Scraper kursów — pełny opis techniczny

P Jak technicznie działa scraper?

`src/lib/scraper.ts` uruchamia Chromium w trybie headless przez Playwright (`chromium.launch({ headless: true, args: ['--disable-blink-features=AutomationControlled'] })`) i dla każdego bukmachera otwiera osobny kontekst przeglądarki z polskim locale (`pl-PL`) i strefą czasową `Europe/Warsaw` , własnym user-agentem desktopowym i wstrzykniętym skryptem maskującym `navigator.webdriver` (żeby strona nie wykryła automatyzacji). Żądania obrazów, mediów i fontów są blokowane (`context.route`) — potrzebny jest tylko DOM. Strona jest otwierana z `waitUntil: 'domcontentloaded'` , timeout 35 sekund, po czym scraper czeka 4 sekundy, próbuje kliknąć jeden z dziewięciu możliwych przycisków akceptacji cookies, a następnie wykonuje auto-scroll (5× przewinięcie o 1200px z pauzą 600ms), żeby załadować rekordy doładowywane leniwie (lazy-loaded).

P Ile stron jest scrapowanych w jednym przebiegu i jak rozłożone jest obciążenie?

Scraper definiuje 16 zadań (`tasks`): 5 dedykowanych ekstraktorów dla piłki nożnej (Superbet, STS, Fortuna, Totalbet, Etoto), 5 generycznych dla piłki nożnej (Fuksiarz, Betclic, forBET, LvBet, Betfan) i kolejne 6 dla koszykówki (Superbet, STS, Fortuna, Etoto, Fuksiarz, Betclic) — łącznie 16 wizyt na stronach. Funkcja `runPool(tasks, 6)` ogranicza liczbę równoległe otwartych stron przeglądarki do 6, żeby nie przeciążyć maszyny hostującej ani nie wzbudzić podejrzeń u bukmacherów przez nagły skok ruchu z jednego adresu IP.

P Jakie konkretne strategie ekstrakcji DOM są używane dla każdego bukmachera?

Bukmacher	Strategia	Szczegóły
STS	<code>extractSts</code>	Dedykowane selektory <code>.one-ticket-match-tile</code> , osobne elementy na nazwę gospodarzy/gości (wersja mobilna i desktopowa), kursy z przycisków oznaczonych „1”/„X”/„2”.
Superbet	<code>extractSuperbet</code>	<code>.event-card</code> , nazwy zespołów z dedykowanych elementów <code>.event-team-name</code> , kursy odczytywane parami etykieta-wartość („1” , „X” , „2” + liczba).
Fortuna, Totalbet	<code>extractWynikMeczu</code>	Szuka w karcie wydarzenia sekcji „Wynik meczu” (piłka) lub frazy zawierającej „zwycięzca” (koszykówka, tryb dwudrogowy z dogrywką) i czyta linie tekstu w ustalonej kolejności względem nagłówka rynku.
Etoto	<code>extractEtoto</code>	Selektor <code>li.single-event</code> ; nazwy zespołów to pierwsze dwie „sensowne” linie przed godziną meczu, kursy to liczby po godzinie.

Fuksiarz, Betclit, forBET, LvBet, Betfan	<code>extractGenericRows</code>	Generyczny ekstraktor próbujący listy typowych selektorów wierszy meczu (<code>[class*="event-row"]</code> , <code>[class*="match-row"]</code> itd.), rozpoznaje nazwy zespołów po wzorcu „A - B” / „A – B” / „A vs B” albo po dwóch najbliższych liniach przed pierwszym kursem, odróżnia kursy od dat o podobnym formacie liczbowym (np. „11.06” vs „11.06” jako kurs) sprawdzając, czy linia zaraz po niej wygląda jak godzina.
--	---------------------------------	---

P Jak scraper rozpoznaje, że dwa wiersze u różnych bukmacherów to ten sam mecz?

Funkcja `matchTeams` normalizuje nazwy zespołów (`normalizeName`): małe litery, usunięcie polskich znaków diakrytycznych i akcentów (NFD + usunięcie znaków combining), usunięcie typowych skrótów klubowych („FC”, „CF”, „AC”, „FK”, „KS”, „GKS” itd.) i spójników. Dodatkowo działa słownik `NAME_ALIASES` dla całych nazw (np. „usa” → „stany zjednoczone”, różne formy „DR Kongo”/„DR Konga”) oraz `TOKEN_ALIASES` dla skrótowych form słów („pld” → „południowa”). Po normalizacji nazwy są dzielone na tokeny (słowa o długości >2), a dwie nazwy uznaje się za ten sam zespół, gdy co najmniej 65% tokenów jednej zawiera się (lub jest zawarte) w tokenach drugiej.

P Jak działa scalanie (klastrowanie) wydarzeń z wielu bukmacherów w jedno?

Funkcja `mergeResults` przechodzi po wszystkich wynikach scrapowania i dla każdego wiersza próbuje znaleźć istniejący „klastery” (potencjalnie ten sam mecz) o tym samym sporcie, gdzie nazwy zespołów się zgadzają (wprost lub odwrotnie — gdy bukmacher zapisał gospodarza i gościa w innej kolejności, kursy są odpowiednio zamieniane). Mecze o tych samych nazwach, ale których odczytany czas startu różni się o więcej niż 6 godzin (`MAX_KICKOFF_DIFF_MS`), są traktowane jako różne wydarzenia (np. mecz młodzieżowy i seniorski o tej samej nazwie). Druga, finalna przebiegowa faza scalania (`coalesced`) łączy z powrotem klastry, które rozjechały się tylko z powodu niedokładnego parsowania daty/godziny u jednego z bukmacherów (np. „dziś”/„jutro” bez roku), a nie dlatego, że to faktycznie inny mecz.

P Jak liczony jest czas rozpoczęcia meczu z tekstu typu „jutro 21:00”?

`parseCommenceTime` rozpoznaje słowa „jutro” (dodaje 1 dzień), „dziś”/„dzisiaj” (zostaje przy dzisiejszej dacie) lub wzorzec daty `dd.mm[.rrrr]` , a następnie nakłada na to godzinę z wzorca `hh:mm` . Wynik jest zwracany jako pełny ISO timestamp.

P Jak generowane jest stabilne ID wydarzenia?

ID ma format `scraped-{sport}-{slug(gospodarz)}-vs-{slug(gość)}` , gdzie `slug` to znormalizowana nazwa zespołu z myślnikami zamiast spacji, ograniczona do 40 znaków. Gdy dwa różne wydarzenia wygenerowałyby identyczne ID (np. ten sam mecz drugiej ligi powtórzony w innym terminie), dodawany jest numeryczny sufix (`-2` , `-3` ...). Dzięki temu ID nie zależy od żadnego ulotnego identyfikatora API i pozostaje takie samo między kolejnymi przebiegami scrapera, o ile nazwy zespołów się nie zmieniają — linki w aplikacji i wpisy w dzienniku nie wygasają po odświeżeniu danych.

P Jakie filtry wiarygodności stosuje scraper, zanim dane trafią do silnika arbitrażu?

- `isValidEvent` odrzuca wiersze bez nazwy gospodarza/gościa, z identyczną nazwą obu stron, z nazwą dopasowaną do wzorca śmieciowego (`GARBAGE_TEAM_NAME` — „Bet Builder”, „boost”, „bonus”, „promocja”, „zakład”, „kupon” — czyli promocyjne wiersze, które czasem trafiają się na liście meczów), z kursem poza zakresem 1.01–66, albo z sumą implikowanych prawdopodobieństw poza realnym zakresem marży bukmachera (0.92–1.45 dla rynków dwudrogowych, 0.95–1.6 dla 1X2).
- Funkcja `getBookmakerMargins` dodatkowo odrzuca obliczone marże poza zakresem [-2%, 25%] jako artefakty parsowania.
- Silnik arbitrażu (`MAX_PLAUSIBLE_PROFIT_PERCENT = 20`) odrzuca każdą "okazję" z obliczonym zyskiem powyżej 20% jako niemożliwą w praktyce.
- `data-service.ts` wymaga co najmniej 2 różnych bukmacherów na wydarzenie (`MIN_BOOKMAKERS_PER_EVENT`) — wydarzenie pokryte przez tylko jednego bukmachera nie nadaje się do porównania ani arbitrażu, więc jest odfiltrowywane jako szum.

P Czy scraper łąpie też linki do konkretnego zakładu u bukmachera?

Tak, ale ostrożnie: `eventUrl` jest ustawiany tylko wtedy, gdy sparsowany element wiersza jest sam fizycznym znacznikiem `<a href= "... ">` na stronie — scraper nigdy nie "zgaduje", który zagnieżdżony link w wierszu jest właściwy, bo błędne zgadnięcie wysłałoby użytkownika na złą stronę. Gdy takiego linku nie da się jednoznacznie ustalić, interfejs używa zamiast tego strony głównej bukmachera.

6. Silnik arbitrażu — algorytmy i podatek

P Jak dokładnie liczony jest podatek 12% od zakładów wzajemnych?

To podatek od obrotu (od stawki), nie od wygranej — `netOddsAfterTax(odds, taxRate) = odds × (1 - taxRate/100)`. Przykład z kodu: stawka 100 zł przy kursie 1.10 brutto — do gry faktycznie wchodzi 88 zł netto, co przy tym samym kursie zwraca $88 \times 1.10 = 96.80$ zł, czyli stratę, mimo że kurs brutto jest powyżej 1. Funkcja odwrotna `grossOddsForNet` przelicza docelowy kurs netto z powrotem na to, co trzeba zobaczyć u bukmachera brutto.

P Dlaczego Betclic jest pomijany w przeliczeniu podatkowym?

Betclic oferuje tryb gry bez podatku (pokrywa go sam) — lista `TAX_FREE_BOOKMAKER_KEYS` w `store-types.ts` zawiera obecnie tylko `'betclic'`, ale jest rozszerzalna, jeśli inni bukmacherzy wprowadzą analogiczne promocje. Funkcja `isTaxFreeBookmaker` normalizuje klucz bukmachera (małe litery, bez spacji) przed porównaniem z tą listą.

P Czy użytkownik może zmienić stawkę podatkową?

Tak — `UserSettings.taxRate` (domyślnie 12%) jest edytowalne w panelu konta i przekazywane jako parametr do wszystkich funkcji silnika arbitrażu (`findArbitrageForEvent`, `findAllNearArbitrages` itd.), które domyślnie korzystają z `DEFAULT_TAX_RATE` tylko jeśli nie podano innej wartości.

P Jakie inne ustawienia użytkownika wpływają na wykrywanie okazji?

- `minProfitForAlert` — minimalny % zysku, który wywołuje powiadomienie o surebecie (domyślnie 0.5%).
- `preSurebetThreshold` — margines ponad 100% uznawany za "blisko progu" (domyślnie 2, czyli do 102% sumy implikowanych prawdopodobieństw).
- `roundingEnabled` / `roundingStep` — zaokrąglanie proponowanych stawek do najbliższej wielokrotności (np. 10 zł) przez funkcję `roundStake`.
- `excludeNicheMarkets` / `mainMarketsOnly` — filtrowanie rynków niszowych (na razie aplikacja obsługuje tylko h2h, więc te flagi są przygotowane na przyszłość, gdy przybędą inne typy rynków).
- `selectedBookmakers` — lista bukmacherów, u których użytkownik faktycznie ma konto; domyślnie 9 z 10 scrapowanych (poza brakującym w domyślnej liście jednym kluczem).

7. System powiadomień i alertów

P Jakie typy alertów obsługuje aplikacja?

- **Alerty "mecz się zaczyna"** (`checkAndSendMatchAlerts`) — globalne ustawienie na koncie użytkownika (`notify_settings` : włączone/wyłączone, liczba minut przed startem, e-mail); worker co minutę sprawdza wszystkie nadchodzące wydarzenia i wysyła e-mail z najlepszymi kursami (h2h) dla tych w oknie czasowym ± 1 minuta od żadanego wyprzedzenia.
- **Alerty per-wydarzenie** (`checkAndSendEventAlerts` , tabela `event_alerts`) — użytkownik może utworzyć subskrypcję powiązaną z konkretnym meczem, trzech typów: `before-kickoff` (X minut przed startem), `at-time` (o konkretnej godzinie zegarowej) lub `odds-threshold` (gdy kurs wybranego wyniku u jakiegokolwiek bukmachera przekroczy zadaną wartość). Po wysłaniu alertu wpis jest usuwany z bazy (jednorazowe wyzwolenie).
- **Powiadomienia w aplikacji** (`/notifications`) — wewnętrzna historia zdarzeń: nowy surebet, surebet bliski progu, wygasła okazja, zakończony zakład; w obecnej wersji to dane demonstracyjne (`DEMO_NOTIFICATIONS`) — nie ma jeszcze realnego sprzężenia z silnikiem arbitrażu generującego te wpisy automatycznie.

P Jak technicznie wysyłany jest e-mail?

Aplikacja sama nie wysyła e-maili — buduje payload (`MatchAlertPayload` : dane wydarzenia, czas do startu, najlepsze kursy per wynik z linkiem do bukmachera) i wysyła go jako POST na `N8N_WEBHOOK_URL` (zewnętrzny workflow n8n, skonfigurowany osobno, odpowiedzialny za faktyczne wysłanie e-maila). Jeśli zmienna środowiskowa nie jest ustawiona, funkcja `sendToN8n` zwraca błąd bez próby wysyłki.

P Jak unikane jest wielokrotne wysłanie tego samego alertu?

Dla alertów "mecz się zaczyna" lokalny plik `src/lib/notified-events.json` przechowuje ostatnie 500 ID wydarzeń, dla których alert już wysłano (`readNotifiedIds` / `markNotified`) — raz oznaczone wydarzenie nie wywoła drugiego powiadomienia, nawet dla innego użytkownika w tej samej rundzie sprawdzania. Dla alertów per-wydarzenie ochronę przed duplikatem zapewnia samo usunięcie wiersza z tabeli `event_alerts` po wysłaniu.

P Skąd worker bierze dane o kursach, jeśli działa niezależnie od serwera Next.js?

Czyta bezpośrednio plik `src/lib/scraped-data.json` z dysku (`readScrapedEvents`) — ten sam plik, do którego zapisuje harmonogram scrapingu. Worker i serwer SSR muszą więc współdzielić ten sam katalog na VPS, żeby dane były aktualne dla obu procesów.

8. Warstwa danych (Supabase) i bezpieczeństwo

P Do czego konkretnie służy Supabase?

- Autentykacja użytkowników: e-mail/hasło oraz Google OAuth (`GoogleAuthButton.tsx` , `/auth/callback`).
- Tabela `journal_entries` — dziennik zakładów użytkownika.
- Tabela `notify_settings` — globalne ustawienia powiadomień e-mail per użytkownik (włączone, liczba minut przed startem, adres e-mail).
- Tabela `event_alerts` — subskrypcje alertów powiązane z konkretnym wydarzeniem.

Sam silnik arbitrażu i scraper nie potrzebują bazy danych do działania — operują na pliku JSON.

P Jak rozróżniane są klienci Supabase w kodzie?

Trzy warianty w `src/lib/supabase/` : `client.ts` (po stronie przeglądarki, używa publicznego anon key), `server.ts` (po stronie serwera w komponentach Next.js, operuje na cookies sesji), `admin.ts` (`createAdminClient` , klucz `service_role` — używany wyłącznie przez worker alertów i logikę serwerową, która musi czytać/pisać dane bez kontekstu zalogowanego użytkownika, np. wszystkie ustawienia powiadomień naraz).

P Jak zabezpieczony jest dostęp do danych w bazie?

Row Level Security jest włączone na tabeli `journal_entries` z polityką ograniczającą widoczność wierszy do `auth.uid() = user_id` — każdy użytkownik widzi tylko swój własny dziennik. Analogiczny mechanizm chroni `notify_settings` i `event_alerts` (dostęp przez klucz `user_id`), z wyjątkiem operacji administracyjnych workera, który łączy się kluczem `service_role` (obchodzi RLS z założenia, bo działa poza kontekstem pojedynczego użytkownika).

P Jaki jest pełny schemat tabeli `journal_entries` ?

```
create table journal_entries (  
  id                uuid primary key default gen_random_uuid(),  
  created_at        timestampz not null default now(),  
  user_id           uuid not null references auth.users(id) on delete cascade,  
  event_name        text not null,  
  sport_title       text not null,  
  commence_time     timestampz not null,  
  market_key        text not null,  
  bets              jsonb not null,          -- tablica JournalBet[]  
  total_stake       numeric not null,  
  guaranteed_return numeric not null,  
  profit            numeric not null,
```

```
profit_percent numeric not null,  
status         text not null default 'pending', -- pending|won|lost|cancelled  
notes          text not null default '',  
arbitrage_id   text,  
is_demo        boolean not null default false  
);
```

9. API aplikacji

P POST /api/sync

Uruchamia scraper i nadpisuje `scraped-data.json`. Chronione cooldownem (domyślnie 2 minuty `SCRAPE_COOLDOWN_MIN`) — jeśli dane są świeże, zwraca je bez ponownego scrapowania. Zwraca HTTP 429, jeśli scraping jest już w toku (globalna flaga współdzielona z harmonogramem automatycznym, żeby nie odpalić dwóch przebiegów Playwright naraz).

```
// sukces  
{ "success": true, "count": 87, "timestamp": 1718800000000 }  
  
// cooldown aktywny  
{ "success": true, "skipped": true, "count": 87, "timestamp": 1718799000000,  
  "message": "Kursy są aktualne (odświeżono 1 minutę temu)." }
```

P GET /api/sync

Zwraca status synchronizacji bez uruchamiania scraperów:

```
{ "isSyncing": false, "lastSyncTime": 1718800000000, "lastSyncCount": 87, "hasData":  
true }
```

P GET /api/events/list

Zwraca listę wszystkich wydarzeń jako JSON (bez przeliczania arbitrażu) — używane przez komponenty frontendowe do filtrowania i sortowania po stronie klienta bez ponownego żądania do serwera przy każdej zmianie filtra.

Żaden z tych endpointów nie jest dostępny w wersji statycznej (eksport FTP) — `build-static.mjs` chowa cały katalog `src/app/api` na czas builda, bo route handlers nie mogą zostać wyeksportowane jako statyczne pliki.

10. Zmienne środowiskowe

Zmienna	Przeznaczenie
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL projektu Supabase — wymagane do logowania i dziennika.
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Publiczny klucz Supabase (przeglądarka).
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Klucz administracyjny — używany przez worker alertów i logikę serwerową.
<code>ODDS_API_KEY</code>	Opcjonalny klucz do The Odds API — gdy brak, aplikacja korzysta ze scrapera/danych demo.
<code>SCRAPE_INTERVAL_MIN</code>	Co ile minut harmonogram automatyczny odpala scraper (domyślnie 480 = 8h, minimum wymuszone na 30).
<code>SCRAPE_COOLDOWN_MIN</code>	Minimalny czas między ręcznymi wywołaniami <code>POST /api/sync</code> (domyślnie 2 minuty).
<code>N8N_WEBHOOK_URL</code>	Adres webhooka n8n, na który wysyłane są payloady alertów e-mail.
<code>N8N_AUTH_HEADER</code>	Opcjonalny nagłówek autoryzacyjny dla webhooka n8n.
<code>STATIC_EXPORT</code> / <code>NEXT_PUBLIC_STATIC_EXPORT</code>	Ustawiane wyłącznie przez <code>build-static.mjs</code> — przełącza Next.js w tryb <code>output: "export"</code> .

11. Stos technologiczny — pełna lista

Warstwa	Technologia
Framework	Next.js 15/16 (App Router), SSR + opcjonalny statyczny eksport
Język	TypeScript
Stylowanie	Tailwind CSS
Autentykacja + baza	Supabase (PostgreSQL, Auth, Row Level Security)
Scraper	Playwright (Chromium headless)
Automatyzacja powiadomień	n8n (webhook, wysyłka e-mail)
Zarządzanie procesami (VPS)	PM2 — serwer SSR (port 3002) oraz osobny worker alertów
Hosting frontu	Hostido (shared hosting, statyczny eksport przez FTP) lub VPS (pełny SSR)
Skrypty narzędziowe	<code>scripts/build-static.mjs</code> , <code>scripts/scrape-and-save.ts</code> , <code>scripts/test-scraper.ts</code> , <code>scripts/alerts-worker.ts</code>

Kluczowe pliki logiki: `src/lib/arbitrage.ts` (matematyka arbitrażu), `src/lib/scrapper.ts` (zbieranie kursów, ~1070 linii), `src/lib/data-service.ts` (łączenie scraper/API z arbitrażem, filtrowanie, marże), `src/lib/journal-service.ts` i `alerts-service.ts` (CRUD na Supabase), `src/lib/match-alerts.ts` i `notify-store.ts` (alerty i integracja n8n), `src/instrumentation-node.ts` (harmonogramy w tle).

12. Co udało się zrobić (zakres działający)

P Jakie funkcje są w pełni gotowe i działające?

- **Scraper kursów** dla 10 polskich bukmacherów (STS, Superbet, Fortuna, Betclit, Etoto, Fuksiarz, forBET, LvBet, Betfan, Totalbet), z dedykowanymi i generycznymi strategiami ekstrakcji, anty-detekcją (maskowanie `navigator.webdriver`, blokowanie zasobów medialnych, losowy realistyczny user-agent), poolingiem zadań ograniczającym równoległość do 6 stron.
- **Scalanie wydarzeń między bukmacherami** — normalizacja nazw, aliasy, dopasowanie tokenowe $\geq 65\%$, dwuetapowe klastrowanie (z rozróżnianiem meczów oddalonych czasowo o $>6h$ i ponownym scalaniem rozjazdów wynikających z błędów parsowania daty).
- **Stabilne ID wydarzeń**, niezmiennie między kolejnymi przebiegami scrapera.
- **Pełny silnik arbitrażu**: wykrywanie surebetów, rozkład stawek proporcjonalny, podatek 12% (z wyjątkiem Betclit), filtr 20% maks. zysku, watchlista near-arbitrage z wyliczonym kursem docelowym, kalkulator marż bukmacherów, funkcja hedgingu.
- **10 stron frontendowych**: Dashboard, Arbitraż, Wydarzenia (lista + szczegóły), Kalkulator, Marże, Konto (6 zakładów), Powiadomienia, Grupy MŚ 2026, Strategie (Value Bet + Kelly Criterion), Roadmapa.
- **Autentykacja** (e-mail/hasło + Google OAuth) i dziennik zakładów z RLS w Supabase.
- **System alertów e-mail** trójtrowy: globalne powiadomienie przed startem meczu, per-wydarzenie (3 typy wyzwalaczy), integracja z n8n, deduplikacja wysyłki.
- **Tryb demo** w pełni nawigowalny bez konfiguracji (brak scraped-data.json i brak ODDS_API_KEY → statyczne dane realistyczne).
- **Dwa tryby wdrożenia** — pełny serwer SSR pod PM2 oraz statyczny eksport do hostingu FTP, ze skryptem automatycznie przełączającym konfigurację i przywracającym stan źródłowy.
- **Automatyczny harmonogram scrapingu** (instrumentation hook) i endpoint ręcznej synchronizacji z cooldownem i ochroną przed równoległym uruchomieniem.

P Czego się nauczyliśmy / co zadziałało lepiej niż zakładano?

- Generyczny ekstraktor (`extractGenericRows`) sprawdził się dla większości serwisów opartych na podobnych wzorcach DOM (Fuksiarz, Betclit, forBET, LvBet, Betfan) — nie trzeba pisać osobnego parsera dla każdego z nich.
- Dopasowanie tokenowe nazw drużyn ($\geq 65\%$) okazało się odporne na różnice w zapisie nazw klubów, bez budowania pełnego słownika aliasów dla każdej drużyny.
- Rozdzielenie hostingu statycznego (tani FTP) od logiki serwerowej (VPS) pozwoliło uniknąć kosztownego hostingu Node.js dla samego frontu, przy zachowaniu pełnej funkcjonalności alertów i scrapingu na osobnym, mniejszym procesie.

13. Co nie udało się zrobić / czego zabrakło

P Jakie funkcje z roadmapy zostały odłożone lub nie weszły?

Zgodnie z `src/app/roadmap/page.tsx`, w pełni zrealizowana jest tylko **Faza 1 (MVP)**. Pozostałe fazy:

- **Faza 2 — dane na żywo:** WebSocket/SSE, auto-refresh co 30s, powiadomienia push, zaawansowane filtrowanie/sortowanie, dodatkowe rynki (over/under, handicap) — **niezrealizowane**.
- **Faza 3 — szersze scrapowanie:** Betfair Exchange, bukmacherzy zagraniczni (Pinnacle, Bet365, Unibet, Betway), rotacja proxy, cache Redis/Upstash, rate limiting — **niezrealizowane**. Bukmacherzy zagraniczni są przygotowani w strukturze (`ALL_BOOKMAKERS`), ale nie są scrapowani — silniejsze mechanizmy antybotowe i/lub wymóg logowania.
- **Faza 4 — analiza zaawansowana:** historia kursów i trendy, predykcja ML, rozliczenia wielowalutowe — **niezrealizowane**. Wyjątkiem jest kalkulator Kelly Criterion / Value Bet — to jedyny element tej fazy, który już działa (strona `/strategies`), zrealizowany przed pozostałymi punktami fazy.
- **Faza 5 — monetyzacja:** plan premium, bot Telegram/Discord, publiczne API — **częściowo** (konta i dziennik istnieją, flaga `isPremium` w ustawieniach jest przygotowana, ale brak realnego systemu płatności, bota czy publicznego API).
- **Faza 6 — produkcja i skalowalność:** CDN, monitoring (Sentry/LogRocket), pełne testy automatyczne, CI/CD — **niezrealizowane**. Redis/cache pośredni także nie wdrożony — choć dwutorowy model wdrożenia (SSR + worker PM2) jest krokiem w stronę skalowalności poza pierwotny plan jednego serwera.

P Czy próbowano scrapować bukmacherów zagranicznych?

Tak, są przygotowani w strukturze danych (`ALL_BOOKMAKERS`) zawiera Pinnacle, Bet365, Unibet, Betway, William Hill, Marathon Bet, bwin, Betsson, 1xBet, DraftKings, FanDuel, BetMGM, PZBuk), ale scraping się nie powiódł na tyle skutecznie, by włączyć je do produkcji — ich strony stosują silniejsze mechanizmy antybotowe i/lub wymagają zalogowania, co wykracza poza zakres prostego scrapera DOM bez rotacji proxy i bez systemu zarządzania sesją.

P Czy jest cache albo kolejka zadań?

Nie ma Redis, kolejki zadań ani warstwy cache pośredniego — każde żądanie strony z kursami w trybie SSR odczytuje plik `scraped-data.json` i liczy arbitraż na żywo. To prostsze w utrzymaniu, ale ogranicza skalowalność przy większym ruchu (każde żądanie powtarza tę samą pracę obliczeniową).

P Co jest jeszcze niedokończone w samym systemie powiadomień?

Lista w `/notifications` obecnie korzysta z danych demonstracyjnych (`DEMO_NOTIFICATIONS`) — backend alertów e-mail (worker, n8n) działa naprawdę, ale wewnętrzna historia powiadomień w aplikacji nie jest jeszcze w pełni podłączona do realnych zdarzeń silnika arbitrażu (np. automatyczny wpis "nowy surebet" przy każdym przebiegu scrapera).

14. Główne ograniczenia projektu

P Jakie jest największe ograniczenie techniczne?

Prędkość odświeżania kursów. Scraper oparty na Playwright + Chromium headless musi fizycznie otworzyć każdą z 16 stron bukmacherskich, poczekać na załadowanie (do 35s timeout + 4s + auto-scroll ~3.4s), zaakceptować cookies i sparsować DOM, nawet z ograniczeniem do 6 równoległych kontekstów. Pełny przebieg trwa około **2–3 minuty**. To oznacza, że:

- nie da się odświeżać kursów w czasie rzeczywistym (sekundy) — domyślny interwał automatyczny to 8 godzin (`SCRAPE_INTERVAL_MIN=480`), a ręczne odświeżenie ma cooldown 2 minuty właśnie po to, by nie przeciążać stron bukmacherów ciągłym scrapowaniem i nie narazić się na zablokowanie adresu IP;
- między dwoma odświeżeniami kursy u bukmacherów mogą się zmienić wielokrotnie — okno wykrycia surebetu może się zamknąć, zanim użytkownik zdąży zareagować.

P Czy surebety wykrywane przez aplikację są zawsze aktualne?

Nie zawsze. Dane są tak świeże, jak ostatni scraping (do 8h, lub mniej jeśli użytkownik ręcznie odświeżył z zachowaniem cooldownu). W zakładach sportowych okazje arbitrażowe bywają krótkotrwałe (minuty), więc realna szansa na „złowienie” surebetu zanim kurs się skoryguje jest ograniczona przez częstotliwość scrapingu, a nie przez samą logikę wykrywania, która jest matematycznie poprawna i działa na danych w momencie ich zebrania.

P Czy łatwo jest znaleźć prawdziwe surebety?

Nie — to drugie kluczowe ograniczenie. Nawet z poprawnie działającym scraperem i dobrą logiką wykrywania, **rzeczywistych surebetów na rynku polskim jest mało i są nietrwałe**. Powody:

- Bukmacherzy szybko korygują kursy, gdy widzą jednostronny napływ zakładów (to właśnie tworzy i zamyka okazje arbitrażowe).
- Polscy bukmacherzy często kopiują się nawzajem cenowo (podobne marże, podobne typowania), więc rozjazdy kursów wystarczające do arbitrażu po uwzględnieniu 12% podatku są rzadkie.
- 12% podatek od obrotu znacząco zawęży próg — kurs musi być rozjeżdżony dość mocno, żeby po przeliczeniu na netto nadal wyszedł zysk (kurs musi przekroczyć ok. 1.136, żeby po podatku zwrócić więcej niż wpłacono), co dodatkowo redukuje liczbę realnych okazji.
- Filtr wiarygodności (odrzućcie okazje >20% zysku) słusznie eliminuje błędy parsowania, ale tym samym eliminuje też dużą część „okazji”, które i tak były fałszywe.
- Wymóg co najmniej 2 bukmacherów na wydarzenie (`MIN_BOOKMAKERS_PER_EVENT`) zawęży pole jeszcze bardziej, bo nie każdy mecz jest pokryty przez wszystkich 10 scrapowanych serwisów naraz.

P Jak zaadresowano te dwa ograniczenia na poziomie produktu?

Zdecydowano się **przesunąć nacisk z „czystego łowienia surebetów” na pokazywanie najwyższych dostępnych kursów** u poszczególnych bukmacherów dla danego wydarzenia/rynku:

- Widok porównawczy w `/events` pokazuje kursy wszystkich bukmacherów obok siebie dla każdego wydarzenia, z naciskiem na to, który bukmacher daje **najlepszy kurs** na dany wynik — nawet jeśli nie składa się to w pełny surebet (typ `OutcomeComparison.bestOdds` jest wyliczany dla każdego wyniku niezależnie od arbitrażu).
- Watchlista (near-arbitrage) jest świadomym kompromisem: skoro pełne surebety są rzadkie i krótkotrwałe, użytkownikowi pokazuje się też okazje „blisko progu” (różnica <3%, mecz w ciągu 72h), żeby miał szansę zareagować, gdy kurs jeszcze trochę się ruszy.
- Tabela marż (`/margins`) pozwala użytkownikowi samodzielnie ocenić, który bukmacher ma generalnie najlepsze (najniższe marżowo) kursy w danej dyscyplinie, niezależnie od tego, czy akurat trwa surebet.
- Strona `/strategies` z kalkulatorem Value Bet i Kelly Criterion daje alternatywną strategię zarabiania (obstawianie kursów "z wartością" względem własnego oszacowania), niezależną od dostępności surebetów.
- System alertów e-mail (próg kursowy, wyprzedzenie czasowe) pozwala użytkownikowi reagować na zmiany bez konieczności ciągłego odświeżania strony, częściowo kompensując rzadkość scrapingu.
- W praktyce aplikacja działa więc jako **narzędzie do znajdowania najlepszych kursów i okazji granicznych**, a nie wyłącznie „maszynka do gwarantowanego zysku” — bardziej realistyczne pozycjonowanie biorąc pod uwagę ograniczenia częstotliwości danych i rzadkość prawdziwych surebetów.

P Jakie są inne, mniejsze ograniczenia?

- **Rynki ograniczone do dwu-/trójdrogowych** — piłka nożna (1X2) i koszykówka (zwycięzca z dogrywką). Brak handicapów, over/under, rynków szczegółowych — wymagałoby znacznie bardziej złożonego parsowania DOM dla każdego bukmachera.
- **Brak historii kursów** — przechowywany jest tylko ostatni stan (`scraped-data.json`), bez bazy historycznej do analizy trendów lub trenowania modeli predykcyjnych.
- **Zależność od struktury DOM bukmacherów** — każda zmiana układu strony może zepsuć ekstraktor i wymaga ręcznej aktualizacji selektorów; typowe ryzyko każdego scraper'a bez publicznego API.
- **Brak testów automatycznych** — zwiększa ryzyko regresji przy zmianach w logice arbitrażu lub scraperze.
- **Brak cache/Redis** — przy większym ruchu obliczanie arbitrażu na żywo przy każdym żądaniu może obciążyć serwer.
- **Wersja statyczna jest zamrożonym zdjęciem danych** z momentu builda — jeśli ktoś korzysta z wersji hostowanej na Hostido przez FTP, kursy nie odświeżają się same; trzeba wykonać nowy build i wgrać go ponownie.

- **Historia powiadomień w aplikacji nie jest jeszcze podłączona do realnych zdarzeń** — działa na danych demo, mimo że sam mechanizm wysyłki e-mail jest produkcyjny.

15. Decyzje produktowe i ich uzasadnienie

P Dlaczego podatek 12% jest wliczany do kalkulacji, a nie tylko pokazywany informacyjnie?

Kurs „powyżej 1.0” może mimo to generować stratę po podatku od obrotu. Pokazywanie tylko kursów brutto sugerowałoby fałszywe okazje. Dlatego wszystkie wyliczenia finansowe (stawki, zysk, próg arbitrażu) bazują na kursie netto, a kursy brutto są wyświetlane tylko jako odniesienie do tego, co widać u bukmachera.

P Dlaczego Betclit jest wyjątkiem podatkowym?

Betclit oferuje tryb gry bez podatku (pokrywa go sam), więc jego kursy nie są dyskontowane w obliczeniach. Lista wyjątków jest rozszerzalna, jeśli inni bukmacherzy wprowadzą podobne promocje.

P Dlaczego odrzucane są okazje z zyskiem >20%?

Tak wysoki „zysk” w realnym arbitrażu bukmacherskim praktycznie nie istnieje — oznacza w 99% przypadków błąd scrapowania (pomyłone mecze, źle sparsowany kurs, nieaktualna strona). Filtr chroni użytkownika przed postawieniem pieniędzy na podstawie danych będących artefaktem błędu technicznego.

P Dlaczego cooldown 2 minuty na ręczne odświeżenie?

Żeby nie bombardować stron bukmacherów częstymi automatycznymi wizytami, co groziłoby zablokowaniem adresu IP scrapera i utratą dostępu do danych w ogóle. To kompromis między świeżością danych a stabilnością mechanizmu zbierania danych.

P Dlaczego wymagane są przynajmniej 2 bukmacherzy na wydarzenie?

Kursy jednego bukmachera nie da się z niczym porównać — pojedyncze źródło to szum, nie okazja. Filtrowanie takich wydarzeń na poziomie `data-service.ts` (a nie w samym scraperze) pozwala mu zachować surowe dane dla innych celów (np. tabeli marż per bukmacher), podczas gdy widoki porównawcze i arbitrażowe widzą tylko sensowne, porównywalne wydarzenia.

P Dlaczego dane Mistrzostw Świata 2026 są wpisane ręcznie, a nie scrapowane?

Nie istnieje wiarygodne źródło do automatycznego scrapowania wyników turnieju w obrębie tego samego scrapera (który celuje w kursy bukmacherskie, nie w wyniki sportowe), a "zgadywanie" przypisania grup byłoby tym samym problemem co fałszywy arbitraż: prezentowanie danych z dużą pewnością, które mogą być błędne. Bezpieczniej wpisać je raz ręcznie i utrzymywać aktualność świadomie.

P Dlaczego eksport statyczny chowa cały katalog `api` tylko na czas builda, a nie na trwałe?

Bo te same pliki źródłowe muszą obsługiwać oba tryby wdrożenia — pełny serwer SSR (gdzie API musi działać) i eksport statyczny (gdzie API nie może istnieć). Tymczasowe chowanie i przywracanie po buildzie (nawet gdy build się nie powiedzie — blok `finally`) pozwala utrzymać jedną bazę kodu bez rozdzielania jej na dwa oddzielne projekty.

16. Plany na przyszłość

Krótkoterminowo (Faza 2): odświeżanie danych w czasie bliższym rzeczywistemu (cel <30s opóźnień), więcej typów rynków (handicap, over/under), lepsze filtrowanie i sortowanie, powiadomienia push.

Średnioterminowo (Faza 3–4): szersze pokrycie bukmacherów (cel: 20+, w tym zagraniczni i Betfair Exchange — wymaga rozwiązywania problemu blokad antybotowych, prawdopodobnie przez rotację proxy i/lub zarządzanie sesją logowania), cache pośredni (Redis/Upstash), dane historyczne i analiza trendów, dokończenie value bettingu o pełną integrację z resztą aplikacji.

Długoterminowo (Faza 5–6): monetyzacja (realny plan premium z płatnościami, boty powiadomień na Telegram/Discord, publiczne API dla zewnętrznych deweloperów), pełna infrastruktura produkcyjna (testy automatyczne, CI/CD, monitoring typu Sentry/LogRocket, CDN), dopięcie historii powiadomień do realnych zdarzeń silnika arbitrażu.

Cele biznesowe zapisane w samej aplikacji (`/roadmap`): 20+ bukmacherów monitorowanych, <30s opóźnienia danych, 15+ sportów — żaden z tych celów nie jest jeszcze zrealizowany w obecnej wersji (10 bukmacherów krajowych, opóźnienie liczone w godzinach, 2 sporty: piłka nożna i koszykówka).

17. Podsumowanie

BukScan w obecnej formie to w pełni działający MVP rozwiązujący realny, dobrze zdefiniowany problem (porównanie kursów i wykrywanie arbitrażu na polskim rynku bukmacherskim) z poprawną matematyką, solidnym i odpornym na drobne błędy danych scraperem oraz przemyślanym modelem podatkowym. Dwa fundamentalne ograniczenia rynku i technologii — wolne, bezpieczne odświeżanie danych (2–3 minuty na przebieg, cooldowny chroniące przed blokadą IP) oraz naturalna rzadkość i krótkotrwałość prawdziwych surebetów po doliczeniu 12% podatku — sprawiły, że projekt świadomie poszerzył swoją wartość poza "czyste łowienie surebetów": w stronę porównywarki najlepszych kursów, watchlisty okazji granicznych, tabeli marż i narzędzi strategicznych (Value Bet, Kelly Criterion). To pragmatyczne dopasowanie zakresu do realiów rynku, nie porażka pierwotnego celu. Większość zaawansowanych planów (real-time, szersi bukmacherzy, ML, monetyzacja, pełna infrastruktura produkcyjna) pozostaje w fazie planowania i jest jasno opisana w roadmapie aplikacji.